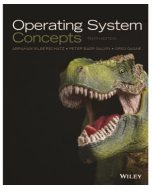


Operating Systems 2024

U3680

National Taipei University

Midterm Exam [Chapter 1 - Chapter 5]

---

1. Answer **True** or **False** for the following statements. **If you answer False, you should give your reason**. For each question, you should write the **question number** (e.g., (a) ___).

   (a) Main memory is the only large storage that the CPU can access directly (3%)
   True. (3%, If you answer No because there are caches, you can get the score)

   (b) The primary goal of multi-programming is to enable interaction. (3%)
   False (1%). Improve CPU utilization or keep the CPU busy. (2%)

   (c) System calls executed in a virtual machine usually have better performance. (3%)
   False (1%). System calls become slower because they require two interrupts: one on the virtual OS (fail) and another on the main OS (success). (2%)

   (d) Message passing is a better choice for processes to communicate through the network. (3%)
   True. (3%)

   (e) To improve portability, system calls are usually implemented by high-level languages. (3%)
   False (1%). System calls are usually implemented using low-level languages due to performance considerations. (2%, alternative: APIs are implemented using high-level languages.)

   (f) Application programming interface (API) is an encapsulation of system calls for user programs. Thus, an API function will contain at least one system call. (3%)
   False (1%). An API function can contain no system calls, such as abs() or pow(). (2%)

   (g) Context switch can improve responsiveness. Thus, it is better to increase the frequency of context switches. (3%)
   False (1%). Context switches are pure overhead. Too frequent context switches will hurt performance. (2%)

   (h) Multiple threads of a process can share the stack memory section. (3%)
   False (1%). The stack memory section contains local variables and function return values. Thus, it cannot be shared. (2%)

   (i) A many-to-one thread model can't support parallelism. Thus, it has no advantages. (3%)
   False (1%). It allows us to share resources and simplify code. (2%)

   (j) Using a one-to-one thread model does not need process-contention scope scheduling. (3%)
   True. (3%)

   (k) When scheduling with a multi-level feedback queue, CPU-bound processes are put in a higher-priority queue. (3%)
   False (1%). CPU-bound processes are put in a lower-priority queue based on SJF. (2%, alternative: IO-bound processes are put in a higher-priority queue.)

   (l) In a busy system, load balancing should be achieved via push migration. (3%)
   False (1%). Using pull migration is a better choice. (2%)

2. Please describe the purposes of **interrupt** and **direct memory access (DMA)**. (6%)

   - The goal of interrupts is to provide an event-driven way to change the control flow in the CPU, so it doesn't have to be busy waiting for the slow devices (3%).
   - DMA allows us to transfer blocks of data without bothering the CPU (or with only one interrupt when the job is done, 3%)

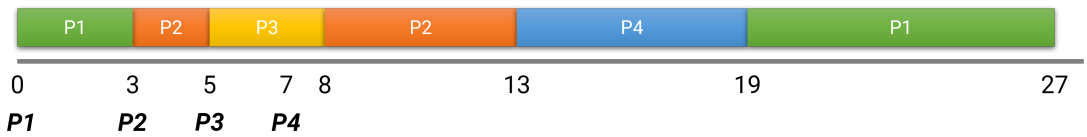
3. Please use an example of a memory access procedure to describe how **caching** works. Does caching always improve data access time? State your reason. (6%)
   When accessing memory, we first search the data in the cache. If the data is found, we can shorten the loading time because accessing the cache is faster. If not found, the data has to be loaded from memory. This time, we copy the data to the cache to speed up the later access (3%).
   Caching only works when the data has a locality. If the data is not used multiple times (e.g., big data), caching only increases overhead (3%).

4. Why a **micro-kernel** approach is usually **safe** but has **poor performance** (6%)
   The micro-kernel approach keeps the Kernel as small as possible (fewer bugs). It is also safer because most parts of the original kernels are moved to user space. Thereby, a crash of these system programs will not lead to a crash of the entire system (3%). However, communications between components require message passing, which is slow. (3%)

5. The following processes are being scheduled using the **preemptive shortest-job-first (SJF)** algorithm:

Process	Arrival	Burst
P_1	0	11
P_2	3	7
P_3	5	3
P_4	7	6

   ...cheduling order of the processes using a **Gantt chart**. (6%)



   (b) What is the **waiting time** for each process? (4%)
   P1: 16; P2: 3; P3: 0; P4: 6. (1% for each answer)

6. Please answer the following questions using **Amdahl's Law**:

   (a) What is the speedup gain of using 8 cores for an 80% parallel program? (4%)
   3.33. (4%)

   (b) If we want to achieve 2 times speedup, how many cores should we use for a 60% parallel program? (4%)
   6 cores. (4%)

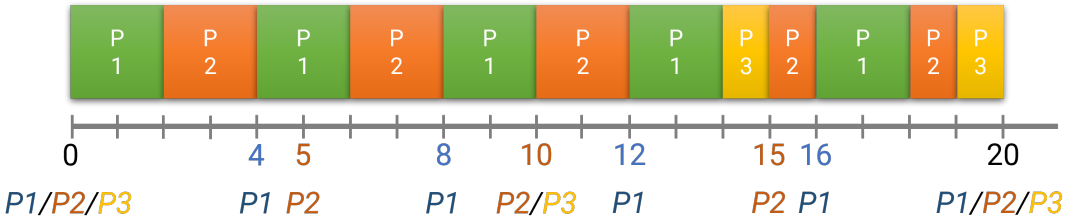
$$\frac{1}{0.2 + \frac{0.8}{8}}$$

7. Please describe the **four** occasions that a **preemptive** scheduling algorithm can re-schedule a process to the running state. (8%)

   - When a process switches from running to waiting state. (2%)
   - When a process switches from running to ready state. (2%, or context switch)
   - When a process switches from waiting to ready state. (2%, **it is also fine if you answer from new to ready state, both result in new candidates.**)
   - When a process terminates. (2%)

8. The following **real-time** processes are being scheduled using the **rate-monotonic (RM)** algorithm. Please show the scheduling order of the first 20 time units using a **Gantt chart**. (6%)

Process	Ready	Execution	Period
P_1	0	2	4
P_2	0	2	5
P_3	0	2	10



9. What are the **process state transitions** during long-term (job) scheduling, short-term (CPU) scheduling, and medium-term scheduling, respectively? Your answers should be: from X state to Y state. (6%)

   - Long-term scheduling: from New state to Ready state (2%)
   - Short-term scheduling: from Ready state to Running state (2%)
   - Medium-term scheduling: from Ready state to Waiting state (2%)

10. Given the program shown below, what are the outputs of this program? (8%)
2 12 16 Program Finished
1 6 8 Program Finished
Each correct number earns 1%. The correctness of "Program Finished" earns 2%.
Get 4% if both "numbers" and "program finished" are correct, but the two lines are in reverse order.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

#define SIZE    3
int arrays[SIZE] = {1, 6, 8};

int main() {
        int i;
        pid_t pid;
        pid = fork();
        if (pid == 0) {
                for (i = 0 ; i < SIZE ; ++i) {
                        arrays[i] *= 2;
                        printf("%d  ", arrays[i]);
                }
        }
        else if (pid > 0) {
                wait(NULL);
                for (i = 0 ; i < SIZE ; ++i)
                        printf("%d  ", arrays[i]);
        }
        printf("Program Finished\n");
        return 0;
}
```

i=0 1 * 2
i=1 6 * 2
i=2 8 * 2